🎤 Sunum Rehberi — QR Menü Otomasyonu

Bu doküman sadece senin içindir. Her teknik terimi, ne yaptığını ve neden yaptığını açıklar.

🔵 Mavi kutu = Sunumda ne söyleyeceğin 🟣 Mor kutu = Ne göstereceğin 🟡 Sarı kutu = Terim açıklaması

---

1 Giriş — Proje Nedir? ~2 dk

Söyle: "Projemizin adı QR Menü Otomasyonu. Bir kafe veya restoranda müşterinin masasındaki QR kodu telefonuyla okutarak dijital menüyü görmesini, ileride sipariş vermesini, mutfak ve kasanın bu siparişi takip etmesini sağlayan bir sistem geliştiriyoruz."

Söyle: "Projede kullandığımız teknolojiler:

- **ASP.NET Core 8.0 MVC** — Microsoft'un web framework'ü. MVC dediğimiz Model-View-Controller mimari deseniyle çalışıyor.
- **PostgreSQL** — Açık kaynaklı veritabanı. Biz Supabase üzerinden kullanıyoruz ki ücretsiz ve cloud tabanlı.
- **Entity Framework Core** — C# nesnelerimizi otomatik olarak veritabanı tablolarına dönüştüren bir ORM aracı.
- **Serilog** — Uygulamamızdaki olayları (hata, bilgi) dosyaya kaydeden loglama kütüphanesi.
- **QRCoder** — QR kod görseli üreten bir NuGet paketi.

"

📖 **Terim — MVC (Model-View-Controller):**
Yazılımı 3 katmana ayıran bir mimari desen:
• **Model** = Veri (veritabanı tabloları, C# sınıfları). Örn: Masa, Urun, Siparis
• **View** = Kullanıcının gördüğü ekran (HTML sayfaları). Örn: Menü sayfası, Admin paneli
• **Controller** = Kullanıcının isteğini karşılayan iş mantığı. Örn: "QR okutuldu → token üret → menüye yönlendir"
Avantajı: Her katman bağımsız değişebilir. Menü tasarımını değiştirmek, veritabanını etkilemez.

📖 **Terim — ORM (Object-Relational Mapping):**
Veritabanında tablo ve satır kavramlarını C# sınıfı ve nesne olarak kullanmamızı sağlar.
`SELECT * FROM Urunler WHERE AktifMi = true` yerine C#'ta

`_context.Urunler.Where(u => u.AktifMi)` yazarız.
Entity Framework Core bunu arka planda SQL'e çevirip veritabanına gönderir.

📖 **Terim — NuGet Paketi:**
.NET dünyasındaki hazır kütüphane deposu. Python'daki pip, JavaScript'teki npm gibi. `dotnet add package QRCoder` diyerek projeye ekliyoruz. Başkasının yazdığı kodu tekrar yazmadan kullanıyoruz.

---

2 Katmanlı Mimari — Proje Nasıl Organize? ~2 dk

🖥️ **Göster:** VS Code'da soldan dosya ağacını aç. `src/` klasörünün altındaki 3 projeyi göster.

Söyle: "Projeyi 3 katmana ayırdık. Her katmanın ayrı bir sorumluluğu var:"

Katman 1: QRMenu.Core (Domain — Projenin Kalbi)

Bu katmanda hiçbir dış kütüphane yok. Sadece saf C# sınıfları var.

- **Entities (13 adet)** — Her biri veritabanındaki bir tabloya karşılık gelen C# sınıfı.
- **Enums (3 adet)** — Sabit değer listeleri. Örneğin sipariş durumu: "Sepette, Onaylandı, Hazırlanıyor..."
- **Interfaces (2 adet)** — Servislerin "ne yapacağını" tanımlayan sözleşmeler. Kodun kendisi burada yok, sadece imza.

📖 **Terim — Entity:**
Veritabanındaki bir tablonun C# karşılığı. Örneğin `Masa` entity'si = veritabanındaki `Masalar` tablosu.
Her property (özellik) bir sütuna karşılık gelir: `MasaNo`, `AktifMi`, `QrKodUrl` gibi.

📖 **Terim — Interface (Arayüz):**
Bir sınıfın hangi metodları içermesi gerektiğini tanımlar, ama nasıl çalışacağını söylemez. Örneğin `ITokenService` der ki: "Bu serviste `GenerateToken()` ve `ValidateTokenAsync()` olmalı."
Asıl kod `TokenService.cs`'de yazılır. Bu sayede istediğimiz zaman farklı bir implementasyon yapabiliriz.

Katman 2: QRMenu.Data (Veri Erişim)

Veritabanıyla konuşan her şey burada:

- **QRMenuDbContext.cs** — Entity Framework'ün ana sınıfı. 13 tablonun tanımı ve ilişki kuralları burada.
- **SeedData.cs** — Proje ilk çalıştığında veritabanına eklenen örnek veriler (5 masa, 10 ürün vs.)
- **Services/** — İş mantığı: `TokenService` (oturum yönetimi), `UrunService` (ürün CRUD)
- **Migrations/** — Veritabanı şemasındaki değişikliklerin kaydı

> 📖 **Terim — DbContext:**
> Entity Framework Core'un kalbi. Uygulamamız ile veritabanı arasındaki köprü.
> Her `DbSet<Masa>` tanımı veritabanında bir tabloya karşılık gelir.
> `_context.Masalar.Add(yeniMasa)` dediğimizde EF Core bunu `INSERT INTO Masalar ...` SQL'ine çevirir.

> 📖 **Terim — Fluent API:**
> Entity'ler arasındaki ilişkileri ve veritabanı kurallarını `DbContext` içinde zincirleme metodlarla tanımlama yöntemi.
> Örneğin: `builder.HasKey(uo => new { uo.UrunId, uo.OpsiyonId })` → iki sütunlu birleşik anahtar (composite key) oluşturur.
> *Bunu Data Annotations (`[Key]` gibi etiketler) ile yapamazdık çünkü iki sütunlu key desteklenmiyor.*

> 📖 **Terim — Migration:**
> Veritabanı şemasını koddan yönetme yöntemi. `dotnet ef migrations add InitialCreate` dediğimizde EF Core entity'lerdeki değişiklikleri analiz eder ve SQL komutları üretir.
> `dotnet ef database update` ile bu SQL'i veritabanına uygularız (tablo oluşturma, sütun ekleme vs.)

> 📖 **Terim — Seed Data:**
> Veritabanı ilk oluşturulduğunda otomatik eklenen test/örnek veriler. 5 masa, 3 kategori, 10 ürün hazır geliyor ki hemen test edebilelim.

Katman 3: QRMenu.Web (Sunum — MVC)

- **Controllers/** — Kullanıcı isteklerini karşılayan sınıflar (AdminController, MenuController, QRController)
- **Views/** — HTML sayfaları (Razor syntax ile C# + HTML birlikte yazılır)
- **Middleware/** — Her HTTP isteğinde otomatik çalışan ara katman (güvenlik kontrolü)
- **wwwroot/css/** — Sayfaların görünümünü belirleyen stil dosyaları

📖 **Terim — Middleware:**
Her HTTP isteği sunucuya geldiğinde, Controller'a ulaşmadan önce sırayla çalışan ara katmanlar.
Bizim `TokenValidationMiddleware`'imiz: "Bu istek geldi → cookie'de token var mı? → DB'de geçerli mi? → Süresi dolmamış mı?" kontrolü yapar. Geçersizse isteği reddeder, geçerliyse Controller'a iletir.

📖 **Terim — Razor View (.cshtml):**
ASP.NET'in şablon motoru. HTML içine `@` işaretiyle C# kodu yazabiliyoruz.
Örneğin: `@foreach (var urun in Model) { <div>@urun.Ad</div> }`
Sunucu tarafında çalışır, müşteriye saf HTML gönderilir.

Söyle: "Bu katmanlı yapının avantajı şu: Veritabanını değiştirmek istesek (örneğin PostgreSQL'den MySQL'e geçsek), sadece Data katmanını değiştirmemiz yeter. Web ve Core katmanları hiç etkilenmez. Aynı şekilde tasarımı tamamen değiştirmek istesek sadece View dosyalarını değiştiriz."

---

3 Veritabanı Tasarımı ~2 dk

🖥️ **Göster:** Hafta1_Rapor.html → Entity tablosu. Veya Supabase panelini açıp tabloları göster.

Söyle: "13 tablo tasarladık. Bunları gruplarsak:"

Grup 1: Menü & Ürünler

Tablo	Ne İşe Yarıyor?	Önemli Sütunlar
Kategoriler	Ürün grupları (Sıcak İçecekler, Atıştırmalıklar...)	`Ad`, `AdEN` (İngilizce), `SiraNo`
Urunler	Menüdeki her ürün	`Ad`, `Fiyat`, `PopulerMi`, `AktifMi`
Opsiyonlar	Boyut, süt tipi gibi seçenekler	`Ad`, `Grup` (Boyut/Süt), `EkFiyat`
UrunOpsiyonlar	Hangi ürünün hangi opsiyonu var? (M2M ilişki)	`UrunId` + `OpsiyonId` (birleşik anahtar)

📖 **Terim — M2M (Many-to-Many) İlişki:**
Bir ürünün birden fazla opsiyonu, bir opsiyonun birden fazla ürünü olabilir.
Bunu doğrudan yapamazsın, arada bir "köprü tablo" lazım = `UrunOpsiyonlar`.
Örnek: Latte → Küçük, Orta, Büyük, Tam Yağlı Süt, Yağsız Süt (5 opsiyon)
Aynı "Küçük" opsiyonu → Latte, Americano, Filtre Kahve'de de var (3 ürün)

Grup 2: Masalar & Oturumlar

Tablo	Ne İşe Yarıyor?
Masalar	Restoran masaları. Her masanın numarası, QR kodu URL'i ve aktiflik durumu var.
Oturumlar	QR okutulunca açılan güvenli oturum. Token hash'i, süre bilgisi, kullanıldı mı bayrağı.

Grup 3: Sipariş & Ödeme (Henüz kod yazılmadı, tablolar hazır)

Tablo	Ne İşe Yarıyor?
Sepet + SepetDetay	Müşterinin henüz sipariş vermediği ürünler (alışveriş sepeti mantığı)
Siparis + SiparisDetay	Onaylanan siparişler. 9 durumlu state machine ile takip.
Odeme	Ödeme kayıtları: tutar, nakit mi kredi kartı mı?

📖 **Terim — State Machine (Durum Makinesi):**
Siparişin geçebileceği durumları ve geçiş kurallarını belirler.
`Sepette → Onaylandı → Hazırlanıyor → Hazır → Teslim Edildi → Ödendi`
Kural: "Hazır" durumundan "Sepette"ye geri dönemezsin. Sadece tanımlı geçişler yapılabilir.
Bu sayede geçersiz durum değişiklikleri önlenmiş olur.

📖 **Terim — RowVersion (Concurrency Token):**
Sipariş tablosunda her güncelleme yapıldığında otomatik değişen bir kolon.
İki garson aynı siparişi aynı anda güncellemeye çalışırsa, ikincisi hata alır.
Bu "Optimistic Concurrency" denen bir tekniktir — veri bütünlüğünü korur.
Örnek: Garson A siparişi "Hazır" yaptı. Garson B de aynı anda "İptal" yapmaya çalışırsa → RowVersion uyuşmaz → çakışma hatası → tekrar dener.

📖 **Terim — Foreign Key (Yabancı Anahtar):**
İki tablo arasındaki bağlantı. `Urunler.KategoriId` → `Kategoriler.Id`'yi referans gösterir.

Bu sayede "Sıcak İçecekler" kategorisini silmeye çalışırsan, altında ürün varsa veritabanı buna izin vermez.

📖 **Terim — Index:**
Veritabanında sık sorgulanan sütunlara eklenen "hızlı arama" mekanizması.
Kitaptaki "dizin" gibi düşün. `IX_Siparisler_Durum` index'i sayesinde "tüm hazırlanan siparişleri getir" sorgusu saniyeler yerine milisaniyeler sürüyor.

---

4 Güvenli Oturum Sistemi (Hafta 2 Ana Özelliği) ~3 dk

Söyle: "Bu haftanın en önemli konusu güvenlik. Bir müşteri QR kodu okutunca neler oluyor, adım adım göstereyim."

Akış: QR Okutmadan Menüye Kadar

Adım adım anlat:

1. **"Müşteri QR kodu okutunca"** — Telefon `https://qrmenu.burhansahin.com.tr/qr/1` adresine gider. Buradaki `1` masa numarası.
2. **"Sunucumuz bir token üretir"** — Token dediğimiz şey 256-bit (32 byte) tamamen rastgele bir karakter dizisi. Bunu üretmek için `RandomNumberGenerator` kullanıyoruz. Bu, kriptografik olarak güvenli bir rastgele sayı üreteci (CSPRNG). Normal `Random` sınıfı tahmin edilebilir ama CSPRNG tahmin edilemez.
3. **"Token'ı hash'leyip veritabanına kaydediyoruz"** — Token'ın kendisini değil, SHA256 hash'ini saklıyoruz. Neden? Veritabanı hacklenirse bile token'lar çalınamaz çünkü hash'ten orijinal değere geri dönülemez.
4. **"Token'ı müşterinin tarayıcısına cookie olarak yazıyoruz"** — Cookie üzerinde 3 güvenlik bayrağı var:
   - `HttpOnly` = JavaScript bu cookie'yi okuyamaz (XSS koruması)
   - `Secure` = Sadece HTTPS üzerinden gönderilir
   - `SameSite=Strict` = Başka sitelerden gelen isteklerde cookie gönderilmez (CSRF koruması)
5. **"Her istekte middleware kontrol eder"** — Müşteri menüde gezinirken her sayfa isteğinde: cookie'deki token okunur → hash'lenir → veritabanından eşleştirme aranır → süre kontrolü yapılır. Geçerliyse sayfa gösterilir, değilse reddedilir.

6. **"90 dakika süresi var"** — Sliding Expiration: Her işlemde süre sıfırlanır. Müşteri 90 dakika hiçbir şey yapmazsa oturum kapanır.

🖥️ **Canlı Demo:**

1. Tarayıcıda `http://localhost:5177/qr/1` aç
2. Menü sayfasına yönlenecek — "Burası menü sayfası, QR okutuldu ve güvenli oturum açıldı" de
3. F12 (Geliştirici Araçları) → Application → Cookies → `QRMenuToken`'ı göster
4. "Bakın HttpOnly aktif, JavaScript ile okuyamazsınız" de

📖 **Terim — CSPRNG (Cryptographically Secure Pseudo-Random Number Generator):**
Kriptografik olarak güvenli rastgele sayı üreteci. Normal `Random` sınıfından farkı: ürettiği sayılar tamamen öngörülemez.
Parola, token, şifreleme anahtarı gibi güvenlik kritik değerler için zorunludur.

📖 **Terim — SHA256 Hash:**
Herhangi bir veriyi 64 karakterlik sabit uzunlukta bir değere dönüştüren tek yönlü fonksiyon.
"merhaba" → `2cf24dba5fb0a30e...` — Geri dönüşü yoktur.
Aynı girdi her zaman aynı hash'i üretir, ama hash'ten orijinal değeri bulamazsın.
Bu yüzden veritabanında token'ın kendisini değil hash'ini saklıyoruz.

📖 **Terim — XSS (Cross-Site Scripting):**
Saldırganın web sayfasına kötü amaçlı JavaScript kodu enjekte etmesi.
Eğer cookie `HttpOnly` olmazsa, saldırgan `document.cookie` ile token'ı çalabilir.
`HttpOnly` bunu tamamen engeller.

📖 **Terim — CSRF (Cross-Site Request Forgery):**
Kullanıcının bilmeden başka bir siteden bizim sitemize istek göndermesi.
`SameSite=Strict` cookie'nin sadece bizim domain'den gelen isteklerde gönderilmesini sağlar.

📖 **Terim — Rate Limiting:**
Belirli bir süredede yapılabilecek istek sayısını sınırlamak. Bizde 10 istek/dakika.
API'yi spam eden veya brute-force deneyen saldırganı yavaşlatır.

---

5 Admin Paneli (Ekstra — 6. Haftadan) ~2 dk

Söyle: "Planda 6. haftaya ait olan admin panelinin temelini şimdiden attık. Responsive bir layout, sidebar menü ve masa yönetimi var."

🖥️ **Canlı Demo:**

1. `http://localhost:5177/admin` aç
2. Sidebar'ı göster: "Bakın sol menüde Dashboard pasif, Masalar aktif. İleride Ürünler, Siparişler, Personel de eklenecek"
3. "+ Yeni Masa Ekle" butonuna bas → modal açılır → masa numarası gir → Oluştur
4. Oluşan masanın kartında "QR Oluştur" butonuna bas → QR kod belirsin
5. "QR İndir" → PNG dosyası iniyor göster
6. "Sil" → masayı silmeyi göster

Söyle: "Masayı silmeye çalıştığımızda, eğer o masaya ait oturum veya sipariş varsa tamamen silmiyoruz, sadece pasife çekiyoruz. Bu soft-delete dediğimiz bir yaklaşım. Veri kaybını önler."

📖 **Terim — AJAX (Asynchronous JavaScript and XML):**
Sayfa yenilemeden sunucuyla iletişim kurma tekniği. "QR Oluştur" butonuna bastığında sayfa yenilenmez, arka planda sunucuya istek gider, cevap gelince sadece o kart güncellenir.
Biz `fetch()` API'sini kullanıyoruz — modern JavaScript'in AJAX yöntemi.

📖 **Terim — Soft Delete vs Hard Delete:**
Hard Delete: Veritabanından tamamen siler. Geri dönüşü yok.
Soft Delete: Silmez, `AktifMi = false` yapar. Veri korunur, raporlarda kullanılabilir.
Bağlı kayıtları olan masalarda soft delete kullanıyoruz.

📖 **Terim — Responsive Tasarım:**
Sayfanın ekran boyutuna göre otomatik uyum sağlaması. Telefonda sidebar kapanır, hamburger menü butonu çıkar. CSS `@media` kuralları ile yapılır.

---

6 Müşteri Menü Sayfası (Ekstra — 7. Haftadan) ~1 dk

🖥️ **Canlı Demo:**

1. `http://localhost:5177/qr/1` aç
2. Kategori tablarına tıkla: "Tümü → Sıcak İçecekler → Atıştırmalıklar"

3. "Popüler" ve "Tükendi" badge'lerini göster
4. Sağ alt köşedeki 🔔 butonuna tıkla

Söyle: "Mobil öncelikli tasarladık. Koyu tema, kategori filtreleme JavaScript ile sayfa yenilemeden çalışıyor. İleride buraya sepete ekleme, opsiyon seçimi ve sipariş gönderme ekleyeceğiz."

---

7 Unit Testler ~1 dk

🖥️ **Göster:** Terminalde `dotnet test tests\QRMenu.Tests` çalıştır. "16 test geçti" çıktısını göster.

Söyle: "16 unit testim var. 8'i ürün CRUD işlemlerini test ediyor — ürün ekleme, silme, güncelleme, filtreleme. 8'i de token ve oturum işlemlerini — token üretimi, hash doğrulama, süre aşımı, oturum sonlandırma."

📖 **Terim — Unit Test:**
Kodun küçük bir parçasını (genellikle tek bir metodu) izole olarak test etme.
Örneğin: "Ürün ekle → ürünü getir → eklediğim ürünle aynı mı?" kontrol eder.
Otomatik çalışır — her değişiklikten sonra 16 testi 2 saniyede çalıştırıp bir şey bozulmuş mu kontrol edebiliyoruz.

📖 **Terim — CRUD:**
Dört temel veritabanı işlemi: **C**reate (oluştur), **R**ead (oku), **U**pdate (güncelle), **D**elete (sil).
Her servisin temel yapı taşı. `UrunService`'te 8 metot var: AddAsync, GetAllAsync, GetByIdAsync, UpdateAsync, DeleteAsync...

📖 **Terim — InMemory Database:**
Testlerde gerçek PostgreSQL yerine bellekte çalışan sahte veritabanı. Test bittikten sonra otomatik silinir. Hızlıdır ve gerçek DB'ye bağımlılığı kaldırır.

---

8 Sonuç & İlerleme Durumu ~30 sn

Söyle: "Hafta 1 ve 2'nin planındaki tüm maddeleri tamamladık. Ek olarak 6. haftadan admin paneli ve masa yönetimi, 7. haftadan müşteri menü sayfasını da geliştirdik. CSS

> dosyalarını düzenli tutmak için ayırdık, 16 unit testim başarıyla geçiyor.
>
> Sıradaki hafta: **Sepet Sistemi** — müşterinin menüden ürün ekleyip sipariş göndermesi. Sonrasında sipariş motoru ve personel ekranları gelecek."

❓ Muhtemel Sorular — Detaylı Cevaplar

Soru	Detaylı Cevap
"Neden Supabase kullandınız?"	Supabase, ücretsiz PostgreSQL veritabanı sunuyor ve connection pooling (bağlantı havuzu) dahil. Bağlantı havuzu dediğimiz şey, her istekte yeni bir veritabanı bağlantısı açmak yerine hazır bağlantıları tekrar kullanmak — bu performansı artırır. Üretim ortamında kendi sunucumuza da geçebiliriz.
"Sepeti neden veritabanında tutuyorsunuz, Redis olmaz mıydı?"	Redis bir bellek-içi (in-memory) cache sistemi. Çok hızlıdır ama ayrı bir sunucu yönetimi gerektirir. Proje ölçeğimizde (bir restoran) PostgreSQL yeterli hızda. Ayrıca veritabanında tutunca TransactionScope ile sepet→sipariş dönüşümünü atomic (ya tamamen olur ya hiç olmaz) yapabiliyoruz. Redis'te bu kadar kolay olmazdı.
"Fluent API nedir, neden kullandınız?"	Entity framework'te veritabanı kurallarını tanımlama yöntemi. İki yol var: Data Annotations (entity sınıfının üstüne [Key], [Required] gibi etiketler) ve Fluent API (DbContext içinde zincirleme metotlarla). Biz Fluent API kullandık çünkü: (1) İki sütunlu birleşik anahtar (UrunOpsiyon), (2) Cascade delete kontrolü (sipariş silinince ödeme de silinsin ama masa silinmesin), (3) Seed data ekleme — bunlar Data Annotations ile yapılamaz.
"Token neden URL'de değil cookie'de?"	URL'deki token paylaşılabilir — müşteri linki WhatsApp'tan birine gönderirse token çalınır. Cookie'de HttpOnly olarak tutarsak: (1) JavaScript erişemez (XSS koruması), (2) URL paylaşılsa bile cookie gitmez. (3) SameSite=Strict ile başka sitelerden istek gönderilse bile cookie eklenmez.
"HTTPS kullanıyor musunuz?"	Evet. Üretim ortamında Nginx reverse proxy ile Let's Encrypt SSL sertifikası var. Cookie'nin Secure flag'i production'da aktif — yani cookie sadece HTTPS bağlantısında gönderilir. Geliştirme ortamında localhost HTTP olduğu için Secure = false yapıyoruz.

"Deployment (yayınlama) nasıl yapıyorsunuz?"	GitHub Actions CI/CD pipeline var. Her commit'te otomatik build + test çalışır. Linux VDS'e (sanal sunucu) Nginx reverse proxy ile deploy ediyoruz. `dotnet publish` ile production build alıp sunucuya yüklüyoruz.
"Katmanlı mimarinin avantajı ne?"	Sorumlulukların ayrılması (Separation of Concerns). Veritabanını değiştirmek istesek sadece Data katmanını değiştiririz, controller'lar etkilenmez. Tasarımı değiştirmek istesek sadece View'ları değiştiririz. Test yazmak kolay çünkü her katman bağımsız test edilebilir. Interface kullanarak mock (sahte) nesneler oluşturabiliyoruz.
"Aynı masaya aynı anda iki garson işlem yaparsa ne olur?"	Sipariş tablosunda RowVersion sütunu var. Bu bir Optimistic Concurrency mekanizması. İlk güncelleyen kazanır, ikincisi DbUpdateConcurrencyException hatası alır ve "Bu sipariş başka biri tarafından güncellendi, lütfen sayfayı yenileyin" mesajı gösterilir. Veri bütünlüğü korunmuş olur.